

Bagian 1 — Pendahuluan

Praktikum ini membahas cara mengambil komentar publik dari video YouTube secara otomatis menggunakan YouTube Data API v3, kemudian menyimpannya sebagai file CSV, dan melanjutkan ke tahap awal analisis sentimen untuk mengetahui apakah komentar tersebut bersifat positif, netral, atau negatif.

1.1 Tujuan Praktikum

- Memahami cara mengakses YouTube Data API v3 menggunakan Python
- Mengambil komentar (top-level dan balasan) dari satu atau lebih video YouTube secara otomatis
- Menyimpan data komentar ke dalam struktur DataFrame Pandas dan mengekspornya ke file CSV
- Memahami konsep analisis sentimen dan alur pengolahannya
- Menerapkan pra-pemrosesan teks dan penghitungan skor sentimen awal pada komentar

1.2 Apa itu YouTube Data API?

YouTube Data API v3 adalah antarmuka pemrograman (API) yang disediakan oleh Google untuk mengakses data YouTube secara terprogram — termasuk informasi video, saluran, dan komentar. API ini menggunakan sistem kuota: setiap permintaan (request) mengonsumsi sejumlah unit kuota dari total 10.000 unit per hari untuk setiap proyek Google Cloud.

Poin Penting sebelum Memulai

- API Key bersifat rahasia — jangan bagikan atau commit ke repository publik (GitHub, dll.)
- Setiap pemanggilan `commentThreads().list()` mengonsumsi 1 unit kuota dari batas 10.000/hari
- Video dengan komentar dinonaktifkan tidak dapat diambil datanya — akan menghasilkan error
- `maxResults` maksimum per request adalah 100 komentar; untuk lebih banyak komentar gunakan paginasi

Bagian 2 — Persiapan: Mendapatkan API Key

Sebelum menulis kode, kamu memerlukan API Key dari Google Cloud Console. Ikuti langkah berikut:

①

Buka Google Cloud Console

Kunjungi <https://console.cloud.google.com> dan login dengan akun Google. Buat project baru dengan mengklik menu dropdown project di bagian atas, lalu pilih "New Project".

②

Aktifkan YouTube Data API v3

Di menu navigasi kiri pilih APIs & Services → Library. Cari "YouTube Data API v3" dan klik Enable untuk mengaktifkannya di project kamu.

③

Buat Credentials (API Key)

Pergi ke APIs & Services → Credentials → Create Credentials → API Key. Salin API Key yang dihasilkan. Untuk keamanan, klik "Restrict Key" dan batasi penggunaannya hanya untuk YouTube Data API v3.

④

Instalasi Library

Jalankan cell berikut di Google Colab untuk menginstal library yang diperlukan:

```
# Instalasi google-api-python-client
# Library ini menyediakan fungsi build() untuk terhubung ke layanan Google API
!pip install google-api-python-client
```

Bagian 3 — Kode Program: Pengambilan Komentar YouTube

CELL 1

Import Library

```
import pandas as pd
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
```

Penjelasan Setiap Baris:

Baris Kode	Penjelasan
<code>import pandas as pd</code>	Mengimpor library Pandas untuk membuat dan memanipulasi DataFrame (struktur tabel data). Alias "pd" digunakan sebagai singkatan standar.
<code>from googleapiclient.discovery import build</code>	Mengimpor fungsi build() dari library google-api-python-client. Fungsi ini digunakan untuk membuat objek koneksi ke layanan API Google (dalam hal ini YouTube).
<code>from googleapiclient.errors import HttpError</code>	Mengimpor kelas HttpError untuk menangkap error yang muncul saat request ke API gagal, misalnya video tidak ditemukan atau kuota habis.

CELL 2

Fungsi Pengambilan Komentar

```
def video_comments(video_id):
    replies = [] # List kosong untuk menampung semua
    komentar
    youtube = build('youtube', 'v3',
                    developerKey=api_key) # Buat koneksi ke YouTube API

    # Request pertama ke endpoint commentThreads
    video_response = youtube.commentThreads().list(
        part='snippet,replies', # 'snippet' = data komentar utama
        videoId=video_id, # ID video yang komentarnya diambil
        maxResults=100 # Ambil maks 100 komentar per halaman
    ).execute() # Kirim request dan simpan respons

    while video_response: # Loop selama masih ada halaman
        for item in video_response['items']: # Iterasi tiap thread komentar

            # — Ambil data komentar utama (top-level comment) —
            published = item['snippet']['topLevelComment']['snippet']['publishedAt']
            user = item['snippet']['topLevelComment']['snippet']['authorDisplayName']
            comment = item['snippet']['topLevelComment']['snippet']['textDisplay']
            likeCount = item['snippet']['topLevelComment']['snippet']['likeCount']
            replies.append([published, user, comment, likeCount])

            # — Ambil balasan (replies) jika ada —
            replycount = item['snippet']['totalReplyCount']
            if replycount > 0 and 'replies' in item:
                for reply in item['replies']['comments']:
                    published = reply['snippet']['publishedAt']
                    user = reply['snippet']['authorDisplayName']
```

```
        repl      = reply['snippet']['textDisplay']
        likeCount = reply['snippet']['likeCount']
        replies.append([published, user, repl, likeCount])

# — Cek apakah masih ada halaman berikutnya —
if 'nextPageToken' in video_response:
    video_response = youtube.commentThreads().list(
        part='snippet,replies',
        videoId=video_id,
        pageToken=video_response['nextPageToken'], # Token halaman berikutnya
        maxResults=100
    ).execute()
else:
    break # Tidak ada halaman lagi, keluar loop

return replies # Kembalikan semua komentar
```

Penjelasan Konsep Utama:

Konsep / Elemen	Penjelasan
<code>build("youtube", "v3", developerKey=...)</code>	Membuat objek klien YouTube API. Parameter pertama = nama layanan, kedua = versi API, developerKey = API Key yang sudah dibuat di Google Cloud Console.
<code>commentThreads().list()</code>	Endpoint API untuk mengambil daftar thread komentar. Satu thread = satu komentar utama beserta semua balasannya.
<code>part='snippet,replies'</code>	'snippet' berisi metadata komentar (teks, waktu, penulis, like). 'replies' menyertakan balasan dalam respons yang sama (efisien, kurangi jumlah request).
<code>videoId=video_id</code>	ID unik video YouTube. Ditemukan di URL: <code>youtube.com/watch?v=VIDEO_ID</code> — bagian setelah <code>v=</code>
<code>maxResults=100</code>	Batas komentar per satu halaman (request). Nilai maksimum yang diizinkan API adalah 100.
<code>.execute()</code>	Mengirimkan request HTTP ke server Google dan mengembalikan respons dalam bentuk dictionary Python.
<code>nextPageToken</code>	Token yang disediakan API jika masih ada halaman komentar berikutnya. Digunakan sebagai parameter <code>pageToken</code> pada request selanjutnya — inilah mekanisme paginasi.
<code>publishedAt</code>	Waktu komentar dipublikasikan dalam format ISO 8601 (contoh: <code>2024-01-15T08:30:00Z</code>).
<code>authorDisplayName</code>	Nama publik pengguna YouTube yang menulis komentar.
<code>textDisplay</code>	Isi teks komentar (dapat mengandung HTML untuk emoji atau formatting).
<code>likeCount</code>	Jumlah "Like" yang diterima komentar tersebut dari penonton lain.

CELL 3

Set API Key & Jalankan Pengambilan Data

```
api_key = 'MASUKKAN_API_KEY_KAMU_DI_SINI' # Ganti dengan API Key milikmu

# Contoh URL video: https://www.youtube.com/watch?v=P3yd4BX9aaU
# ID video = bagian setelah v= pada URL
video_id = 'P3yd4BX9aaU' # Ganti dengan ID video yang diinginkan

# Panggil fungsi – ini akan mengirimkan request ke YouTube API
comments = video_comments(video_id)
comments # Tampilkan hasil (list of lists)
```

Catatan: Simpan API Key di Google Colab menggunakan Secrets (kunci gembok di sidebar kiri) agar tidak terekspos di kode. Akses dengan: `from google.colab import userdata; api_key = userdata.get("YOUTUBE_API_KEY")`

CELL 4

Fungsi Versi Dictionary — Multi-Video Support

Menggunakan dictionary Python (bukan list) untuk menyimpan setiap komentar. Hasilnya lebih rapi dan langsung dapat dikonversi ke DataFrame Pandas. Selain itu, mendukung pengambilan dari beberapa video sekaligus.

```
from googleapiclient.discovery import build
import pandas as pd

# Buat koneksi ke YouTube API (di luar fungsi agar tidak dibuat ulang setiap pemanggilan)
youtube = build('youtube', 'v3', developerKey=api_key)

def video_comments(video_id):
    comments = []
    results = youtube.commentThreads().list(
        part='snippet', # Cukup 'snippet' jika tidak butuh balasan
        videoId=video_id,
        textFormat='plainText' # Kembalikan teks biasa (bukan HTML)
    ).execute()

    while results:
        for item in results['items']:
            # Ambil bagian snippet komentar utama dan simpan sebagai dictionary
            comment_data = item['snippet']['topLevelComment']['snippet']
            comment = {
                'publishedAt': comment_data['publishedAt'],
                'authorDisplayName': comment_data['authorDisplayName'],
                'textDisplay': comment_data['textDisplay'],
```

```

        'likeCount':          comment_data['likeCount']
    }
    comments.append(comment)          # Tambahkan ke list

# Paginasi: lanjut ke halaman berikutnya jika ada
if 'nextPageToken' in results:
    results = youtube.commentThreads().list(
        part='snippet',
        videoId=video_id,
        pageToken=results['nextPageToken'],
        textFormat='plainText'
    ).execute()
else:
    break

return comments

# — Ambil komentar dari beberapa video sekaligus —
video_ids = ['Hk-2uzu5x50']          # Tambahkan ID video lain jika perlu

all_comments = []
for video_id in video_ids:
    comments = video_comments(video_id)
    for comment in comments:
        comment['videoId'] = video_id          # Tandai asal video setiap komentar
    all_comments.append(comment)

# Konversi list of dictionaries ke DataFrame Pandas
df = pd.DataFrame(all_comments,
                  columns=['videoId', 'publishedAt', 'authorDisplayName',
                          'textDisplay', 'likeCount'])

print(df.head)          # Tampilkan 5 baris pertama

```

Perbedaan Versi Dictionary vs Versi List:

Aspek	Versi List (Cell 2)	Versi Dictionary (Cell 4)
Struktur data	List dalam list: [[val1,val2,...]]	List of dict: [{"key":val,...}]
Keterbacaan kode	Kurang jelas (urutan posisi)	Lebih jelas (nama kunci eksplisit)
Ke DataFrame	pd.DataFrame(data, columns=[...])	pd.DataFrame(data) langsung bekerja
textFormat	HTML (emoji = kode HTML)	plainText (teks bersih, lebih mudah diproses)
Multi-video	Satu video per panggilan	Loop untuk banyak video sekaligus

CELL 5 Hitung Total Komentar yang Berhasil Diambil

```

# Tampilkan jumlah total komentar yang berhasil dikumpulkan

```

```
print(f'Jumlah komentar yang diambil: {len(all_comments)}')
```

Fungsi `len()` menghitung panjang list `all_comments`. Setiap elemen dalam list adalah satu komentar. Ini berguna untuk memverifikasi apakah jumlah komentar yang diperoleh sesuai ekspektasi sebelum melanjutkan ke tahap berikutnya.

CELL 6 | Simpan DataFrame ke File CSV

```
# Ekspor DataFrame ke file CSV di direktori kerja Google Colab
df.to_csv('youtube-comments.csv', index=False) # index=False = jangan simpan nomor indeks
```

Parameter	Nilai	Penjelasan
'youtube-comments.csv'	Nama file	Nama file CSV yang akan dibuat di direktori /content/ (root Colab)
index=False	Boolean	Tidak menyertakan kolom indeks (0,1,2,...) Pandas ke dalam file CSV. Tanpa ini, file akan memiliki kolom angka tambahan yang tidak perlu.

CELL 7 | Download File CSV

```
# Unduh file CSV dari environment Colab ke komputer lokal
from google.colab import files
files.download('youtube-comments.csv')
```

Modul `google.colab.files` hanya tersedia di lingkungan Google Colab. Fungsi `files.download()` memicu unduhan file dari server Colab ke browser pengguna. Jika dijalankan di Jupyter Notebook lokal, baris ini dapat dihapus — file sudah tersimpan langsung di sistem lokal.

Bagian 4 — Alur Analisis Sentimen

Berikut adalah gambar alur lengkap proses analisis sentimen komentar YouTube. Diagram berikut menunjukkan empat fase utama: pengambilan data, pra-pemrosesan teks, penghitungan sentimen, dan output visualisasi.

Alur Analisis Sentimen Komentar YouTube

Menggunakan YouTube Data API v3 + TextBlob / VADER

1 PENGAMBILAN DATA

MULAI

Input: API Key + Video ID YouTube

`youtube = build("youtube", "v3", developerKey=api_key)`

Panggil `commentThreads().list()`

`part="snippet", videoId=video_id, maxResults=100`
Ya
halaman berikutnya

Ada `nextPageToken`?

Tidak

Kumpulkan: `publishedAt` · `authorDisplayName` · `textDisplay` · `likeCount`

Simpan ke DataFrame Pandas → ekspor `youtube-comments.csv`

Baca CSV → `df = pd.read_csv("youtube-comments.csv")`

Hapus duplikat · `drop_duplicates()` | Hapus baris kosong

Normalisasi teks: `lower()` · hapus URL · hapus tanda baca · strip emoji

Deteksi bahasa (`langdetect`) → filter komentar berbahasa Indonesia

Hitung skor sentimen → TextBlob / VADER / IndoBERT

Skor Polaritas?

< 0

= 0

> 0

☹️ Negatif

😊 Netral

😊 Positif

Tambahkan kolom sentimen ke DataFrame → ekspor CSV hasil

Visualisasi: Pie Chart distribusi · Bar Chart · Word Cloud

SELESAI

2 PRA-PENROSESAN

3 ANALISIS SENTIMEN

4 OUTPUT & VISUALISASI

4.1 Penjelasan Setiap Fase

Fase Pengambilan Data

Pada fase ini Python terhubung ke YouTube Data API v3 menggunakan API Key.
Komentar diambil secara paginal (100 komentar per request) hingga semua halaman habis.
Data yang dikumpulkan: waktu posting, nama pengguna, teks komentar, dan jumlah like.
Hasil akhir fase ini adalah file CSV yang berisi semua komentar mentah.

Fase Pra-Pemrosesan Teks

Data mentah dari CSV tidak bisa langsung dianalisis — teksnya perlu dibersihkan terlebih dahulu.
Langkah: hapus duplikat → hapus baris kosong → normalisasi (huruf kecil, hapus URL, tanda baca, emoji).
Deteksi bahasa menggunakan library langdetect untuk memfilter komentar non-Indonesia jika diperlukan.
Output: kolom teks baru "clean_text" yang siap diproses model sentimen.

Fase Analisis Sentimen

Setiap komentar dihitung skor polaritasnya menggunakan library pilihan (TextBlob atau VADER).
Skor polaritas berkisar dari -1.0 (sangat negatif) hingga +1.0 (sangat positif), 0 = netral.
Berdasarkan skor tersebut, setiap komentar diberi label: Positif, Netral, atau Negatif.
Untuk bahasa Indonesia, disarankan menggunakan IndoBERT atau kamus sentimen Bahasa Indonesia.

Fase Output & Visualisasi

Kolom label sentimen ditambahkan ke DataFrame dan diekspor ke file CSV baru.
Visualisasi distribusi sentimen menggunakan Pie Chart atau Bar Chart (Matplotlib/Seaborn).
Opsional: Word Cloud untuk melihat kata-kata yang paling sering muncul per kategori sentimen.

4.2 Perbandingan Library Analisis Sentimen

Library	Bahasa	Cara Kerja	Kelebihan	Keterbatasan
TextBlob	Inggris	Kamus (lexicon-based)	Mudah digunakan, cepat	Tidak optimal untuk Bahasa Indonesia
VADER	Inggris	Kamus + aturan heuristik	Bagus untuk teks media sosial	Khusus Bahasa Inggris
IndoBERT	Indonesia	Model Transformer (deep learning)	Akurasi tinggi untuk Bahasa Indonesia	Butuh resource komputasi lebih besar
Kamus Lokal	Indonesia	Kamus sentimen manual	Ringan, dapat dikustomisasi	Akurasi bergantung pada kelengkapan kamus

Untuk praktikum ini kita akan menggunakan TextBlob (karena kemudahan instalasi) dengan catatan bahwa hasilnya paling akurat untuk komentar berbahasa Inggris. Untuk analisis komentar berbahasa Indonesia yang lebih akurat, pertimbangkan untuk menggunakan IndoBERT di tahap lanjutan.

Bagian 5 — Kode Program: Pra-Pemrosesan & Analisis Sentimen Awal

Lanjutkan dari file CSV yang sudah dihasilkan pada Bagian 3. Cell-cell berikut adalah kelanjutan notebook yang sudah ada.

CELL 8

Instalasi Library Sentimen

```
# Instalasi library yang dibutuhkan untuk analisis sentimen
!pip install textblob langdetect

# Download corpus bahasa Inggris untuk TextBlob
import nltk
nltk.download('punkt')          # Tokenizer kalimat dan kata
nltk.download('averaged_perceptron_tagger') # POS tagger untuk analisis TextBlob
```

Library / Paket	Fungsi
textblob	Library analisis teks yang menyediakan fungsi sentimen, terjemahan, dan koreksi ejaan berbasis NLTK.
langdetect	Library untuk mendeteksi bahasa dari sebuah teks. Mendukung 55+ bahasa termasuk Bahasa Indonesia.
nltk (punkt)	Natural Language Toolkit — paket tokenizer yang diperlukan TextBlob untuk memecah kalimat menjadi token kata.
averaged_perceptron_tagger	Model POS (Part-of-Speech) tagging yang digunakan TextBlob untuk menganalisis peran kata (kata kerja, kata benda, dll.) dalam menentukan sentimen.

CELL 9

Muat CSV & Inspeksi Awal

```
import pandas as pd

# Muat file CSV hasil pengambilan data dari Bagian 3
df = pd.read_csv('youtube-comments.csv')

# Inspeksi awal dataset
print('Dimensi dataset:', df.shape)          # (jumlah baris, jumlah kolom)
print()
print('Informasi kolom:')
print(df.info())
print()
```

```
print('5 baris pertama:')
print(df.head())
```

`df.shape` mengembalikan tuple (`n_baris`, `n_kolom`). `df.info()` menampilkan tipe data setiap kolom dan jumlah nilai non-null — berguna untuk mendeteksi missing values sejak awal.

CELL 10

Pembersihan & Normalisasi Teks

```
import re                                # Library ekspresi reguler untuk operasi teks

# — Langkah 1: Hapus baris duplikat —————
sebelum = len(df)
df = df.drop_duplicates(subset=['textDisplay'])    # Hapus komentar yang teks-nya identik
print(f'Duplikat dihapus: {sebelum - len(df)} baris')

# — Langkah 2: Hapus baris dengan teks kosong —————
df = df.dropna(subset=['textDisplay'])            # Hapus baris di mana textDisplay = NaN
df = df[df['textDisplay'].str.strip() != '']       # Hapus komentar yang hanya berisi spasi

# — Langkah 3: Fungsi normalisasi teks —————
def clean_text(text):
    text = str(text)                            # Pastikan bertipe string
    text = text.lower()                         # Ubah ke huruf kecil
    text = re.sub(r'http\S+', '', text)         # Hapus URL (http://... atau https://...)
    text = re.sub(r'<.*?>', '', text)           # Hapus tag HTML (misal: <br>, <b>)
    text = re.sub(r'[\w\s]', '', text)          # Hapus tanda baca dan karakter khusus
    text = re.sub(r'\d+', '', text)             # Hapus angka
    text = re.sub(r'\s+', ' ', text).strip()    # Normalisasi spasi berlebih
    return text

# — Langkah 4: Terapkan fungsi ke kolom textDisplay —————
df['clean_text'] = df['textDisplay'].apply(clean_text)

# Tampilkan perbandingan teks asli vs teks bersih
print(df[['textDisplay', 'clean_text']].head(5).to_string())
```

Penjelasan Pola Regex (`re.sub`):

Pola Regex	Artinya	Contoh yang Dihapus
<code>r'http\S+'</code>	Semua karakter non-spasi setelah 'http'	<code>https://youtu.be/abc123</code>
<code>r'<.*?>'</code>	Apapun di antara tanda < dan > (non-greedy)	<code>
</code> , <code></code> , <code></code>
<code>r'[\w\s]'</code>	Semua karakter KECUALI huruf/angka (<code>\w</code>) dan spasi (<code>\s</code>)	<code>!</code> , <code>?</code> , <code>@</code> , <code>#</code> , <code>(</code> , <code>)</code>

Pola Regex	Artinya	Contoh yang Dihapus
<code>r'\d+'</code>	Satu atau lebih digit angka berurutan	2024, 100, 3
<code>r'\s+'</code>	Satu atau lebih whitespace berurutan (spasi, tab, newline)	"hello world" → "hello world"

CELL 11 Deteksi Bahasa (Opsional)

```
from langdetect import detect, LangDetectException

def detect_language(text):
    try:
        return detect(text) # Kembalikan kode bahasa: 'id', 'en', dll.
    except LangDetectException:
        return 'unknown' # Teks terlalu pendek atau tidak dikenali

# Terapkan deteksi bahasa ke setiap komentar (bisa lambat untuk dataset besar)
df['language'] = df['clean_text'].apply(detect_language)

# Tampilkan distribusi bahasa yang ditemukan
print('Distribusi bahasa:')
print(df['language'].value_counts())

# Filter: simpan hanya komentar berbahasa Indonesia
df_id = df[df['language'] == 'id'].copy()
print(f'\nKomentar bahasa Indonesia: {len(df_id)} dari {len(df)} total')
```

Bagian Normalisasi Kata Tidak Baku & Singkatan

Komentar YouTube mengandung banyak kata tidak baku, singkatan, dan pengulangan huruf ("kenapaaa", "gk mw", "emg") yang akan mengacaukan analisis sentimen jika dibiarkan. Bagian ini menangani masalah tersebut sebelum model sentimen dijalankan.

Contoh Masalah Kata Tidak Baku di Komentar YouTube

- Singkatan : "gk" → "tidak", "yg" → "yang", "emg" → "memang", "utk" → "untuk"
- Pengulangan : "kenapaaa" → "kenapa", "bagussss" → "bagus", "mantaaap" → "mantap"
- Typo umum : "gmn" → "bagaimana", "blm" → "belum", "sdh" → "sudah"
- Bahasa alay : "sy" → "saya", "bs" → "bisa", "krn" → "karena"

CELL 12

Instalasi & Muat Kamus Kata Tidak Baku

Kita menggunakan dua sumber normalisasi yang saling melengkapi:

Sumber	Deskripsi	Jumlah Entri
indoNLP	Library Python yang mengintegrasikan Colloquial Indonesian Lexicon (Salsabila et al., 2018) dan menangani word elongation secara otomatis.	3.592 token unik → 1.742 lemma
Kamus manual (dict)	Kamus tambahan yang dibuat sendiri untuk menangkap singkatan konteks YouTube yang mungkin belum ada di kamus indoNLP (misalnya "subscribe" → "berlangganan").	Dapat diperluas sesuai kebutuhan

```
# Instalasi library indoNLP untuk normalisasi kata bahasa Indonesia
!pip install indoNLP

from indoNLP.preprocessing import replace_slang, replace_word_elongation, pipeline
# replace_slang()          → mengganti kata slang/alay ke bentuk formal
#                          berdasarkan Colloquial Indonesian Lexicon (3.592 token)
# replace_word_elongation() → menghapus pengulangan huruf berlebih
#                          contoh: 'bagussss' → 'bagus', 'kenapaaa' → 'kenapa'
# pipeline()               → menggabungkan beberapa fungsi preprocessing menjadi satu
```

CELL 13

Kamus Kata Tidak Baku Manual (Tambahan)

Kamus ini melengkapi indoNLP untuk kata-kata yang spesifik di konteks komentar YouTube atau yang belum tercakup kamus Salsabila et al.:

```
# Kamus tambahan – tambahkan entri sesuai konteks video yang dianalisis
kamus_tambahan = {
    # — Singkatan umum —
    'gk' : 'tidak',      'ga' : 'tidak',      'nggak': 'tidak',
    'yg' : 'yang',       'utk' : 'untuk',     'krn' : 'karena',
    'emg' : 'memang',    'blm' : 'belum',     'sdh' : 'sudah',
    'gmn' : 'bagaimana', 'dgn' : 'dengan',    'dr' : 'dari',
    'sy' : 'saya',       'bs' : 'bisa',       'msh' : 'masih',
    'hrs' : 'harus',     'trs' : 'terus',     'tp' : 'tapi',
    'jg' : 'juga',       'lg' : 'lagi',       'bgt' : 'banget',
    'bnyk' : 'banyak',   'sbnrny' : 'sebenarnya', 'kyk' : 'seperti',
    # — Konteks YouTube —
    'subscribe' : 'berlangganan',
}
```

```

'subscribe' : 'berlangganan', # typo umum
'like'      : 'suka',
'comment'   : 'komentar',
'upload'    : 'unggah',
'konten'    : 'konten',      # sudah baku, tidak perlu diubah
'youtuber'  : 'kreator',
}

def apply_kamus_manual(text):
    """Ganti kata tidak baku menggunakan kamus manual token per token."""
    tokens = text.split() # Pecah kalimat menjadi daftar kata
    hasil = [kamus_tambahan.get(kata, kata) for kata in tokens]
    #         ↑ jika kata ada di kamus → ganti, jika tidak → pertahankan
    return ' '.join(hasil) # Gabungkan kembali jadi kalimat

```

CELL 14 | Fungsi Normalisasi Lengkap & Terapkan ke DataFrame

```

def normalize_text(text):
    """
    Fungsi normalisasi lengkap dengan tiga lapisan:
    1. Kamus manual → singkatan YouTube & konteks spesifik
    2. replace_word_elongation() → hapus pengulangan huruf (kenapaaa → kenapa)
    3. replace_slang() → ganti kata slang ke formal (Colloquial Lexicon)
    """
    text = str(text)

    # Lapisan 1: kamus manual (dilakukan lebih dulu karena lebih spesifik)
    text = apply_kamus_manual(text)

    # Lapisan 2: hapus pengulangan huruf berlebih
    text = replace_word_elongation(text)
    # 'bagussss' → 'bagus' | 'hahhahaha' → 'hahaha'

    # Lapisan 3: ganti kata slang menggunakan Colloquial Indonesian Lexicon
    text = replace_slang(text)
    # 'emg' → 'memang' | 'gw' → 'saya' | 'lo' → 'kamu'

    return text

# — Terapkan ke kolom clean_text —————
df['normalized_text'] = df['clean_text'].apply(normalize_text)

# — Tampilkan perbandingan sebelum dan sesudah normalisasi —————
print('Perbandingan teks sebelum dan sesudah normalisasi:')
print('=' * 70)
contoh = df[['textDisplay', 'clean_text', 'normalized_text']].head(8)

```

```
for _, row in contoh.iterrows():
    print(f"Asli      : {row['textDisplay'][:60]}")
    print(f"Clean     : {row['clean_text'][:60]}")
    print(f"Normalized: {row['normalized_text'][:60]}")
    print('-' * 70)
```

Urutan Tiga Lapisan Normalisasi

Lapisan 1 — Kamus manual dijalankan PERTAMA karena menangani singkatan spesifik konteks.

Contoh: "subscribe" → "berlangganan" (indoNLP tidak mencakup kata serapan Inggris)

Lapisan 2 — `replace_word_elongation()` dijalankan SEBELUM `replace_slang()` agar kata hasil perpanjangan seperti "mantaaap" sudah menjadi "mantap" sebelum dicari di kamus.

Lapisan 3 — `replace_slang()` menggunakan kamus 3.592 token sebagai jaring terakhir untuk kata-kata alay/gaul yang tersisa setelah dua lapisan sebelumnya.

CELL 15 | Verifikasi Kualitas Normalisasi

```
# Hitung berapa kata yang berhasil dinormalisasi
def hitung_perubahan(original, normalized):
    """Hitung jumlah token yang berubah antara teks asli dan yang dinormalisasi."""
    orig_tokens = str(original).split()
    norm_tokens = str(normalized).split()
    # Bandingkan token yang sama posisinya
    min_len = min(len(orig_tokens), len(norm_tokens))
    berubah = sum(1 for i in range(min_len) if orig_tokens[i] != norm_tokens[i])
    return berubah

df['jumlah_token_berubah'] = df.apply(
    lambda row: hitung_perubahan(row['clean_text'], row['normalized_text']),
    axis=1
)

total_berubah = df['jumlah_token_berubah'].sum()
komentar_berubah = (df['jumlah_token_berubah'] > 0).sum()

print(f'Total token yang dinormalisasi : {total_berubah:,}')
print(f'Komentar yang mengalami perubahan: {komentar_berubah:,} dari {len(df):,} komentar')
print(f'Persentase komentar ternormalisasi: {komentar_berubah/len(df)*100:.1f}%')
```

Catatan: library langdetect menggunakan probabilitas — komentar pendek (<10 karakter) sering salah terdeteksi. Pertimbangkan untuk tidak memfilter bahasa jika mayoritas audiens video adalah penutur Bahasa Indonesia.

```
from textblob import TextBlob

def get_sentiment_score(text):
    """
    Menghitung skor polaritas sentimen menggunakan TextBlob.
    Output: float antara -1.0 (sangat negatif) dan +1.0 (sangat positif)
    """
    blob = TextBlob(str(text))          # Bungkus teks dalam objek TextBlob
    return blob.sentiment.polarity      # Ambil nilai polaritas

def get_sentiment_label(score):
    """
    Mengubah skor numerik menjadi label kategori sentimen.
    """
    if score > 0.05:                    # Threshold positif: di atas 0.05
        return 'Positif'
    elif score < -0.05:                 # Threshold negatif: di bawah -0.05
        return 'Negatif'
    else:                              # Di antara -0.05 dan 0.05
        return 'Netral'

# Terapkan ke DataFrame
df['sentiment_score'] = df['clean_text'].apply(get_sentiment_score)
df['sentiment_label'] = df['sentiment_score'].apply(get_sentiment_label)

# Tampilkan ringkasan hasil
print('Distribusi Sentimen:')
print(df['sentiment_label'].value_counts())
print()
print('Statistik Skor Sentimen:')
print(df['sentiment_score'].describe().round(4))
```

Elemen	Penjelasan
<code>TextBlob(text)</code>	Membuat objek TextBlob dari string teks. Objek ini menyimpan teks dan menyediakan berbagai metode analisis linguistik.
<code>.sentiment.polarity</code>	Properti yang mengembalikan skor polaritas: float dari -1.0 (paling negatif) hingga +1.0 (paling positif). Dihitung berdasarkan kamus kata-kata bersentimen dalam TextBlob.
<code>.sentiment.subjectivity</code>	Properti tambahan: float 0.0–1.0. Mendekati 0 = faktual/objektif; mendekati 1 = pendapat/subjektif (tidak digunakan di sini tapi tersedia).
Threshold 0.05 / -0.05	Batas ambang yang sedikit di luar nol. Ini menghindari pelabelan "Netral" yang terlalu sempit — komentar dengan skor kecil tapi tidak nol persis tetap dianggap netral.

CELL 17

Simpan Hasil Sentimen ke CSV

```
# Tampilkan beberapa contoh komentar beserta label sentimennya
kolom_tampil = ['authorDisplayName', 'textDisplay', 'clean_text',
                'sentiment_score', 'sentiment_label']
print(df[kolom_tampil].head(10).to_string())

# Simpan hasil lengkap (termasuk kolom sentimen) ke file CSV baru
df.to_csv('youtube-comments-sentiment.csv', index=False)
print('\nFile tersimpan: youtube-comments-sentiment.csv')

# Download ke komputer lokal
from google.colab import files
files.download('youtube-comments-sentiment.csv')
```

CELL 18

Visualisasi Distribusi Sentimen (Awal)

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'DejaVu Sans'

# — Hitung distribusi sentimen —————
dist = df['sentiment_label'].value_counts()
warna = {'Positif': '#1E8449', 'Netral': '#B7950B', 'Negatif': '#C0392B'}
colors = [warna.get(k, '#888888') for k in dist.index]

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# — Subplot 1: Pie Chart —————
axes[0].pie(
    dist.values,
    labels=dist.index,
    autopct='%1.1f%%',
    colors=colors,
    startangle=140,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2}
)
axes[0].set_title('Distribusi Sentimen Komentar YouTube',
                  fontsize=12, fontweight='bold', pad=14)

# — Subplot 2: Bar Chart —————
bars = axes[1].bar(dist.index, dist.values, color=colors,
```

```

        width=0.55, edgecolor='white', linewidth=0.8)
for bar in bars:
    axes[1].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.5,
                 str(int(bar.get_height())) ,
                 ha='center', va='bottom', fontweight='bold', fontsize=10)
axes[1].set_title('Jumlah Komentar per Kategori Sentimen',
                  fontsize=12, fontweight='bold')
axes[1].set_xlabel('Kategori Sentimen')
axes[1].set_ylabel('Jumlah Komentar')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

plt.tight_layout()
plt.savefig('sentiment_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

```

plt.subplots(1, 2) membuat satu baris dua kolom grafik dalam satu figure. axes[0] merujuk ke grafik pertama (Pie Chart) dan axes[1] ke grafik kedua (Bar Chart). plt.tight_layout() otomatis merapikan jarak antar grafik agar tidak saling tumpang tindih.

Bagian 6 — Ringkasan Alur Kode & Langkah Selanjutnya

6.1 Ringkasan Seluruh Cell

Cell	Judul	Output Utama
Cell 1	Import Library	Library siap digunakan
Cell 2	Fungsi pengambilan dengan balasan	Fungsi video_comments() versi lengkap
Cell 3	Set API Key & eksekusi	List komentar mentah
Cell 4	Fungsi versi dictionary + multi-video	DataFrame df dengan semua komentar
Cell 5	Hitung total komentar	Output jumlah komentar
Cell 6	Ekspor ke CSV	File youtube-comments.csv
Cell 7	Download CSV ke lokal	File terunduh ke komputer
Cell 8	Instalasi TextBlob & langdetect	Library analisis sentimen tersedia
Cell 9	Muat CSV & inspeksi	DataFrame siap diproses
Cell 10	Pembersihan & normalisasi teks	Kolom clean_text
Cell 11	Deteksi bahasa (opsional)	Filter komentar Bahasa Indonesia
Cell 12	Analisis sentimen TextBlob	Kolom sentiment_score + sentiment_label
Cell 13	Simpan hasil ke CSV	File youtube-comments-sentiment.csv
Cell 14	Visualisasi distribusi sentimen	Pie Chart + Bar Chart sentimen

6.2 Langkah Selanjutnya (Pengembangan Lanjutan)

Analisis Lebih Dalam yang Dapat Dilakukan

- Word Cloud: visualisasi kata-kata paling sering muncul per kategori sentimen (Positif/Netral/Negatif)
- Analisis temporal: tren sentimen berdasarkan waktu posting (publishedAt) — apakah sentimen memburuk setelah event tertentu?
- Korelasi like vs sentimen: apakah komentar positif mendapat lebih banyak like?
- IndoBERT: ganti TextBlob dengan model transformer untuk akurasi lebih tinggi pada Bahasa Indonesia
- Topic Modeling: gunakan LDA (Latent Dirichlet Allocation) untuk menemukan topik utama dalam komentar
- Multi-video comparison: bandingkan distribusi sentimen antar beberapa video berbeda

Keterbatasan yang Perlu Diperhatikan

- Kuota API: 10.000 unit/hari — video dengan ratusan ribu komentar bisa menghabiskan kuota
- TextBlob tidak optimal untuk Bahasa Indonesia — pertimbangkan kamus sentimen lokal
- Komentar yang sangat pendek (emoji saja, angka saja) sering menghasilkan skor tidak akurat
- API tidak menyediakan komentar yang sudah dihapus atau disembunyikan pemilik video